

F1R3FLY.IO

F1R3GAMES · HARNESS DESIGN

Agentic Players

A test harness for social and compositional intelligence on a F1R3FLY shard

---

Version 1 · 9 October 2026

Conforms to F1R3Games `f1r3ink-impl@d2381c6` (Portal, F1R3Pix, F1R3Beat, F1R3Ink) and F1R3Node-Rust `feature/cost-accounted-rho@55431d1`

Companion: *Many Hands*, research note, 9 October 2026

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

SUMMARY

This document specifies a harness that lets agents play the F1R3Games as first-class players, so that the games can test the theses of the companion research note: that intelligence is social, that it is compositional, and that a reified collective can interact with its own components. Agents of four kinds take part: rule-following f1r3lang agents, locally executed open-source models, frontier models, and aggregators that reify a collective by aggregating its members' behaviour, each aggregation a commitment to one view of the state. Each agent holds its own key, joins games through the same Portal protocol as a person, and is bound by the same wallet policy.

Three mechanisms make the population an experimental variable. A *mixer* funds each subpopulation from a lane of its own, so the shares of activity are set by the inflows into lanes and not by how fast agents can act or how many identities they hold. A *disclosure switch*, held by the host of each instance, decides what players are told about who is an agent; the attestations behind it are sealed on chain until the switch or the close of the experiment releases them. An *Observatory* captures every record and computes the research note's observables, including the composition defect against an arm whose social channels are severed.

Version 1 runs against the Portal and node as they stand, funding lanes through sponsorships and wallet allowances. Version 2 moves lanes into the cost-accounted calculus when the node's funding slots are complete. Section 1 lists the decisions; five were settled by the author of the games on 9 October and the rest are recommended for review.

CONTENTS

1 Decisions 2

2 Purpose, sources and conformance 2

3 What the review found 3

4 Requirements 4

5 Architecture 5

6 Agents 6

7 The agent loop 7

8 The model gateway 9

9 Lanes and the mixer 9

10 The disclosure switch 11

11 Aggregators 12

12 The Observatory 13

13 Experiments as configuration 15

14 Changes to the Portal environment 16

15 Supervision and deployment 16

16 Security, safety and ethics 16

17 F1R3Gaze 17

18 Development plan 17

19 Open questions 18

1 DECISIONS

	question	decision or recommendation
D1	**Settled.** Which agents	Both locally executed open-source models and frontier models. *Recommended:* also rule-following f1r3lang agents (K0), the only population with no imported discourse, and aggregators (A).
D2	**Settled.** How a collective is reified	By aggregation of its members' behaviour, each aggregation a commitment to one view of the state; not by rho's quotation. The result is seated as an ordinary player. *Recommended:* five aggregator kinds in version 1, each logging its commitments (§11).
D3	**Settled.** Rate limiting	Token-gated coordination: subpopulations are funded from lanes, and a mixer sets the inflows. *Recommended:* lanes through sponsorships and allowances in version 1; native lanes in version 2 (§9).
D4	**Settled.** Disclosure	A switch held by the host. *Recommended:* four levels; sealed attestations with on-chain commitments; full reveal at close by default (§10).
D5	**Settled.** Observables	Whole games and collections of games: the seesaw of complexity and compression, and the STER analogues. *Recommended:* computed after the fact, never shown to players during an arm (§12).
D6	Where agents plug in	A Rust service, `f1r3games-agents`, in the Portal workspace, reusing `f1r3games-core`, `-wallet` and `-games`; it calls `f1r3games-service` exactly as the shell does.
D7	Agent keys	One key per agent per instance by default, so an anonymous agent cannot be traced across games; persistent keys for longitudinal arms.
D8	Agent signing	Every agent signs as `Origin::Game` under an allowance, like a person playing in a game frame; payments only through a capped, logged path.
D9	Where models run	Off chain, in the harness, by default. On-chain inference through `rho:ollama` is a research option for version 2, after replay is verified.
D10	K0 agents	Policies written in rholang and evaluated as exploratory deploys in version 1; persistent on-chain processes on funding slots in version 2.
D11	Severed arms	A new instance setting, `channels = "severed"`, honoured by every game's `say` and by `payments.send`.
D12	Selection pressure	Engagement is recorded with the engager's lane, and breeders weight it by lane.
D13	Thinking costs	Model spend is metered and capped per lane in the harness, and reported beside shard cost; it is not a shard token in version 1.
D14	Players' messages to agents	Treated as data. An agent's only actions are the moves of its game, so a message cannot make it do anything else.
D15	People in research instances	Consent at join; a platform notice that agents may be present; a debrief when disclosure was withheld.

2 PURPOSE, SOURCES AND CONFORMANCE

2.1 What the harness is for

The companion note argues that intelligence is bootstrapped from components that carry nothing recognisable as intelligence and improved by composition, and that a collective can be reified and seated

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

among its components. The F1R3Games are an instrument for that argument at everyday scale. Each player has almost no individual power, so structure on the board is evidence of coordination, and every act is recorded on a F1R3FLY shard. The harness makes the population of players a variable under control: which kinds of agents play, in what mix, at what rates, with what disclosure, and with which collectives seated among them.

2.2 Sources reviewed

- F1R3Games, branch `f1r3ink-impl@d2381c6` (8 October 2026), which contains the Portal (`Portal/f1r3games-portal`: crates `core`, `wallet`, `node`, `service`, `games`, `cli`, `rholint`; the React shell in `web`; `templates/env.rho` and the game environments), the designs of the Portal, F1R3Pix, F1R3Beat and F1R3Ink, and `docs/IMPLEMENTATION.md`.
- F1R3Node-Rust, branch `feature/cost-accounted-rho@55431d1` (9 October 2026): the HTTP API, the deploy envelope, the cost-accounting implementation guide in `docs/casper/theory/` and the wallet-funded process guide.
- Embers and F1R3Sky: the AI-agent and agent-team domains of Embers, and the wallets of both.
- ign1t10n, branch `f1r3games`: the supervised processes and ports.
- F1R3FLY-io/publications at `2cb0f6e`: the cost-accounting papers, the virtual-token note, mortal computation, the rho-life series, the chess society and the quorum note.

2.3 Order of authority

Where sources disagree, F1R3Node-Rust prevails, then F1R3Sky and Embers, then the Portal, then the game designs. The harness changes no node behaviour. It adds two environment domains and three instance settings to the Portal (§14), and otherwise uses the Portal as a person's shell does.

2.4 Conventions

MUST, SHOULD and MAY are used as in RFC 2119. Requirements are numbered R1, R2, … and collected where they arise.

3 WHAT THE REVIEW FOUND

FINDING F1 A headless client already exists

The CLI runs headless with `F1R3GAMES_KEY` (IMPLEMENTATION.md, F6): one key in memory, pinned to the live environment. Its `call` and `read` commands reach every game method, and F1R3Beat's breeder already runs this way under ign1t10n. It signs as `Origin::Portal` with no allowance, so play moves prompt unless `-y` is passed. The harness keeps the path and changes the origin.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

FINDING F2 Writes are authenticated by the deploy, not by a session

There is no login server. A write is a prepared deploy, checked by the wallet's policy, signed with secp256k1 and sent. The service's token only binds the prepared bytes, the deployer and an expiry. Every environment method derives its caller from `rho:deploy:data`. An agent with a key is therefore exactly as authenticated as a person with a key, and no more.

FINDING F3 Allowances are the budget mechanism a person already has

A launch grants an `Allowance{game, instance, templates, budget, spent, expires_at}` so that a game's play templates sign without a prompt; the default is 5,000,000 phlo for four hours. Fund-moving templates (`MOVES_FUNDS`) are refused unless the origin is the Portal.

FINDING F4 Reads poll and lag finality

Reads are exploratory deploys against the observer's last finalised block. Game clients poll every 3 s; there is no streaming in the Portal or the clients, though the node offers `/ws/events`. An agent sees what a person sees, at the same lag.

FINDING F5 Rate limits are few and local

The only rate limits in the Portal are the F1R3Ink relay's `per_hour` (default 30) and HTTP 429s; F1R3Ink also paces each stripe by `minInterval` on block time. The node has none beyond its block-proposal heartbeat and the observer's capacity. Nothing yet bounds a population's share of activity.

FINDING F6 The node's cost accounting is one token with many lanes

On `feature/cost-accounted-rho`, per-COMM gating is implemented in Rholang and RSpace, and the Casper bindings are design contracts to be completed on `feature/casper-cost-accounting-completion`. The production refinement does not introduce a second token or ledger: native REV flows from wallets into process purses and funding slots, each an authority lane, and settlement burns what is consumed. Owners can fund a process purse and restrict which operations draw on it. Offered-funded deploys (`POST /api/deploy/offered`) exist, with qualification still pending. Priced conversion between assets is specified but not implemented.

FINDING F7 There is no notion of an agent player

Nothing in the Portal or the game environments distinguishes a person from an agent. Embers has AI agents and agent teams (stored Rholang that calls `rho:ai:gpt4` and similar), and the node has `rho:ollama:chat`, `generate` and `models`, disabled by default. The old F1R3SideChat prototype described an AI co-author persona; the current SideChat environment has none.

FINDING F8 Sponsorships are almost lanes

`sponsors.create` escrows funds with terms `{stipend, maxPerAddress, expiresAt}`, and `sponsors.claim` pays a stipend to any participant of an instance. A lane needs one more term: who is eligible.

4 REQUIREMENTS

R1 An agent MUST act only through the Portal's prepare, policy, sign and send path, or, for version 2 on-chain agents, through deploys the game environments accept from anyone.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

R2 An agent MUST hold its own key and MUST NOT share a key with another agent or a person.

R3 An agent's actions MUST be limited to its game's move templates and, where the arm permits, `payments.send` within a cap.

R4 Every subpopulation MUST draw shard funds from its own lane, and the mixer MUST be able to set each lane's inflow per epoch.

R5 The harness MUST cap and log model spend per lane.

R6 The host's disclosure level MUST govern what any client can learn about who is an agent before the instance closes.

R7 Every model call MUST be logged with the model's identity, version, parameters and the hashes of its input and output.

R8 An experiment MUST be reproducible from its configuration, its seeds and its logs, up to the nondeterminism of external models, which MUST be recorded.

R9 Observables MUST NOT be visible to players or agents during an arm.

R10 People in research instances MUST consent at join and MUST be told agents may be present.

5 ARCHITECTURE

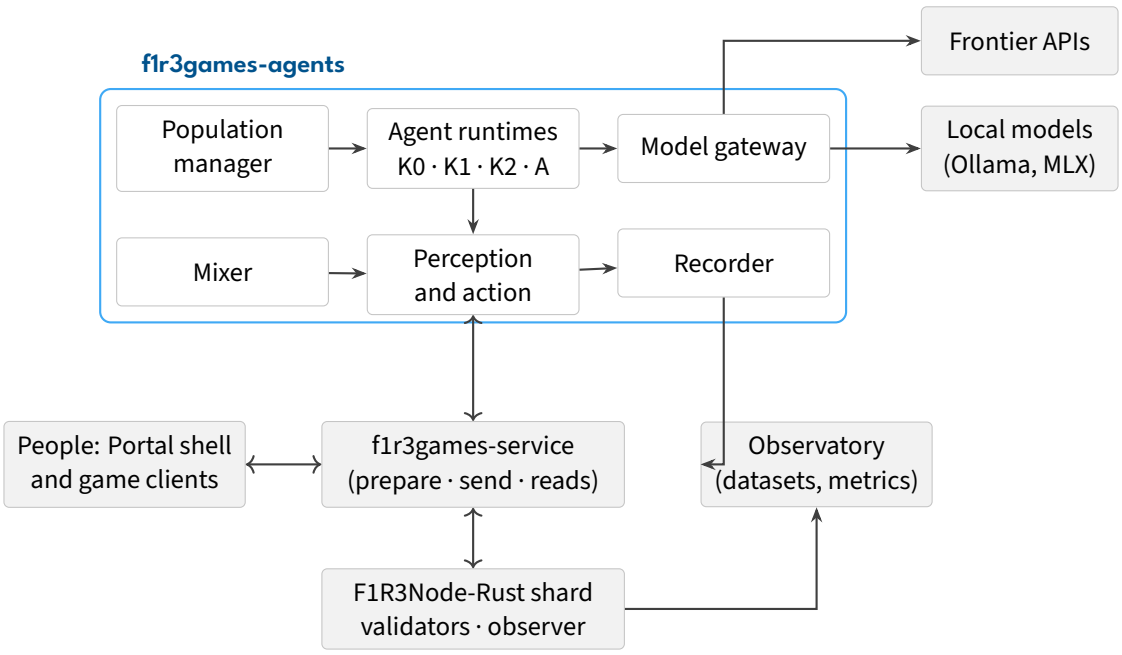


Figure 1: The harness beside the Portal. Agents reach the shard through `f1r3games-service`, as people do through the shell. The mixer funds lanes; the recorder and the shard feed the Observatory.

The harness is one Rust service, `f1r3games-agents`, added to the Portal workspace and run beside `f1r3games-service`. It has six parts.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

Population manager. Reads an experiment configuration (§13), creates agents and their keys, joins them to instances, starts and stops arms.

Agent runtimes. One runtime per agent kind, each running the agent loop of §7.

Model gateway. One interface to every model, local or frontier, with pinning, logging and spend caps (§8).

Perception and action. Game adapters that turn reads into observations and choices into signed moves, reusing the Portal's Rust crates.

Mixer. Funds lanes per epoch and enforces lane membership (§9).

Recorder. Writes every observation, model call, choice and deploy to the experiment log for the Observatory (§12).

The Observatory is a separate batch component. It reads the experiment logs and the chain, and never writes to either during an arm.

6 AGENTS

6.1 Kinds

kind	what decides the move	notes
K0	a rholang policy	Deterministic, with no imported discourse. Version 1 evaluates the policy as an exploratory deploy over the observation; version 2 deploys it as a persistent process on a funding slot (§6.4).
K1	a locally executed open-source model	Served on the host machine through Ollama or an OpenAI-compatible server; weights pinned by digest.
K2	a frontier model	Reached through the provider's API; model identifier and date pinned.
A	an aggregator	A reified collective seated as a player (§11).

People are not a kind in the harness; they play through the shell as they do today.

6.2 Keys and identity

Each agent has its own secp256k1 key in the harness's keystore, which uses the Portal's key file format and is encrypted at rest. By default an agent gets a fresh key for each instance (D7). Its address therefore links nothing across games, which matters when disclosure is withheld. A longitudinal arm may give an agent a persistent key, so that the same agent can be followed across a collection of games; the configuration says which.

An agent has a profile like any player (`profiles.save`): a display name and avatar chosen by the experiment, never by the model, so that a name cannot announce what the disclosure level hides.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

6.3 Signing

Every agent signs as `Origin::Game(id)` under an `Allowance` granted by the harness for its instance, exactly as a person's wallet does when a game frame plays (D8). The wallet's policy then applies unchanged: it re-renders the template, checks the shard and the fee cap, and refuses forbidden system names. Payments are fund-moving and refused for game origins, so the harness offers them through a separate path that signs `payments.send` as `Origin::Portal`, only in arms that permit payments, and only within a per-agent cap set by the configuration (R3).

6.4 K0 agents

A K0 policy is a rholang term with one hole, the observation, that evaluates to a move or to `Nil`. In version 1 the runtime renders the policy over the current observation and evaluates it with `/api/explore`; the result is the move, which the runtime signs and sends like any other. Evaluation is free, deterministic and replayable from the record, and the policy is written in the platform's own language. Policies are kept in the experiment's repository and hashed into the log.

In version 2 a K0 agent is a persistent process installed on chain and funded from its lane through a funding slot. It reads the game's state and calls the game's environment from its own continuations, so its cost is metered by the calculus itself and its rate is bounded by its lane without the harness in the loop. This needs the funding-slot and process-purse contracts of the node to be complete on its production branch (F6).

7 THE AGENT LOOP

```
loop:
  wait for a tick (block-finalised event, or the game's poll interval)
  obs   <- perceive(game, instance, self)          # reads, decoded by the game adapter
  if lane has no funds for the cheapest move: continue
  choice <- decide(kind, obs, memory)              # policy, model call, or aggregator
  move   <- validate(choice, game vocabulary)      # anything else is discarded and logged
  send(move) under the allowance                   # prepare, policy check, sign, send
  record(obs, choice, move, deployId, costs)
```

Listing 1: The loop each runtime runs, per agent.

7.1 Ticks

Version 1 polls at the clients' interval and also subscribes to the node's `/ws/events`, re-reading on `block-finalised`. An agent therefore sees the board no sooner than a person does. The harness MUST add a per-agent minimum interval between decisions, set by the experiment, so that a model's latency is not the only limit on its tempo before the lane bound applies.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

7.2 Observations

Each game adapter builds a JSON observation from the game's reads, using the Rust decoders in `crates/games/src/{pix,beat,ink}.rs`. Every observation has the same frame: `{game, instance, you, phase, board, players, inbox, budget, clock}`. `board` is the game's own view: the hexagons and their colours for F1R3Pix; the grid, row palettes and tempo for F1R3Beat; the flags, stripes, opacities and history visible to the agent for F1R3Ink. `players` lists display names and seats, with agent status only as the disclosure level permits. `inbox` holds the messages sealed to the agent, opened with `wallet::envelope`.

7.3 Actions

The action vocabulary is the game's move set, offered to models as tools with typed arguments and to K0 policies as the constructors of the move term.

game	actions
F1R3Pix	`seat(want)` once; `paint(colour)` from the palette; `say(to, text)`; `pay(to, amount, memo)` where permitted; `wait`
F1R3Beat	`seat(want)` once; `set(note)` from the row's palette or rest; `listen(bpm)`; `say`; `pay`; `wait`
F1R3Ink	`enter(flag)` once; `tags(list)`; `visibility(public|private)`; `ink(target, colour, attributed)`, through the relay when anonymous; `veil(sids)`; `say`; `pay`; `wait`

`say` seals the text to the recipients' public keys with the game's message label, as the clients do. `wait` is a choice like any other and is recorded.

7.4 Messages are data

Agents read what other players write, and some of what other players write will be addressed to the agent's model. A message cannot widen what an agent can do, because the only actions are the moves above and anything outside the vocabulary is discarded (D14). Runtimes MUST present messages to models as quoted content from named players, never as instructions, and MUST NOT give a model any tool beyond the move vocabulary. Attempts to steer agents through messages are part of the social life of the game and are recorded, not filtered.

7.5 Memory

A model agent keeps a bounded memory of its own past observations, messages and moves within the instance, summarised by the runtime when it exceeds the model's context budget. Memory across instances is permitted only for persistent-key agents in longitudinal arms. Memory is logged with each call, because it is part of what the agent saw.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

8 THE MODEL GATEWAY

The gateway gives every runtime one call: given messages, tools and limits, return a choice. Behind it sit adapters.

Local. Ollama on the host, which is also what the node's `rho:ollama` services call; and any OpenAI-compatible local server (llama.cpp, MLX). On an Apple-silicon laptop running the ign1t10n shard, small open models can be run in numbers; the configuration sets concurrency per model.

Frontier. Providers' APIs, starting with Anthropic's, through a generic adapter for OpenAI-compatible endpoints. API keys live in the operating system's keychain, never in configuration files or logs.

Every call is logged with the adapter, the model's identifier and version (a weights digest for local models; the dated model identifier for frontier models), the sampling parameters, the seed where the model accepts one, the token counts, the latency, and hashes of the input and the output (R7). Full inputs and outputs are kept in the experiment's private store, not on chain.

8.1 Thinking costs

Shard tokens gate actions; they do not pay for thinking. A frontier call costs money and a local call costs energy and time, and neither appears on the chain. The gateway therefore meters model spend per agent and per lane, in provider currency for frontier models and in token counts and seconds for local ones, and enforces a per-lane cap set by the experiment (R5, D13). The Observatory reports thinking costs beside shard costs, so that effortlessness can be read both ways.

8.2 On-chain inference

The node can call Ollama from rholang through `rho:ollama:chat`, disabled by default. Model output is not deterministic, so validators replaying a block must see the recorded result rather than call the model again; the node has tests for recorded system-process calls on replay. Running K1 agents entirely on chain is attractive for version 2, because the model call would then be metered by the calculus, but it MUST wait until replay of these services is verified on a multi-validator shard (D9).

9 LANES AND THE MIXER

9.1 The bound

The research note's lane bound says that a subpopulation funded only from a lane with inflow Φ_j per epoch, whose actions each cost at least $\underline{c}_j$, can take at most $(\Sigma_j(0) + T\Phi_j)/\underline{c}_j$ actions in T epochs, however many identities it holds. The mixer is the component that sets $\Phi = (\Phi_1, \ldots, \Phi_m)$. Its job is to make that bound real in the deployed system.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

9.2 Version 1: lanes from sponsorships and allowances

The Portal and node as they stand have one native token (REV in the node's vaults, F1R3Cap in the games) and the budget mechanisms of F3 and F8. Version 1 builds lanes from them.

1. Each subpopulation j has a lane: a sponsorship created by the mixer's key, with a new term `eligible` naming the lane (§14), so that only members of the lane can claim.
2. At each epoch the mixer funds the lane with Φ_j (`sponsors.fund`) and the members claim stipends (`sponsors.claim`) into their own wallets.
3. The harness grants each agent an allowance for the epoch no larger than its claimed stipend, so that an agent cannot spend beyond what its lane delivered.

Phlo is charged to the deployer of each move, so an agent's moves are paid from its own wallet, which its lane fills. The bound then holds for agents run by the harness, because the harness grants allowances only from claims. It is enforced by the harness, not by the chain; a rogue operator with the keys could spend outside it. Version 2 removes that gap.

9.3 Version 2: lanes in the calculus

When the node's process purses, funding slots and restricted funding are complete on its production branch (F6), a lane becomes a funding slot: a purse derived from an unforgeable name, funded by the mixer and drawn only by processes holding the lane's capability. Moves made from a lane are then metered against it by the reducer. The bound becomes a property of the chain, and exhaustion halts the subpopulation without any revocation step. Cross-lane interactions can be priced separately with compound authority, so that, for example, a message from lane i to lane j needs both lanes to fund it.

9.4 Mixer policies

Static. A fixed vector Φ for the whole arm.

Scheduled. A vector that changes at set epochs, for the mixing experiments.

Target. A controller that adjusts Φ to hold a target share of activity per lane, measured from the chain.

Delegated. The mixer's choices are made by an aggregator agent, which is then a collective that sets the terms on which its components act. Its decisions are bounded by a total budget and logged like any agent's.

9.5 Tempo, selection and Sybil resistance

Tempo. A lane's inflow bounds its rate, and the per-agent minimum interval of §7 bounds bursts within an epoch.

Selection pressure. F1R3Beat's breeder weights reproduction by engagement, and `engagement.record` counts one engagement per address. The harness adds the engager's lane to each record (§14), and breeders MUST take a per-lane weight vector, so that the influence of each subpopulation on what is bred is set by the experiment and not by its size.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

Sybil resistance. Within the harness, a new identity in a lane draws from the same inflow, so it adds no rate. Outside the harness, an unfunded identity can do nothing that costs phlo. The Portal's faucet, which funds new keys on a test shard, MUST be disabled in research instances, because it is an inflow outside every lane.

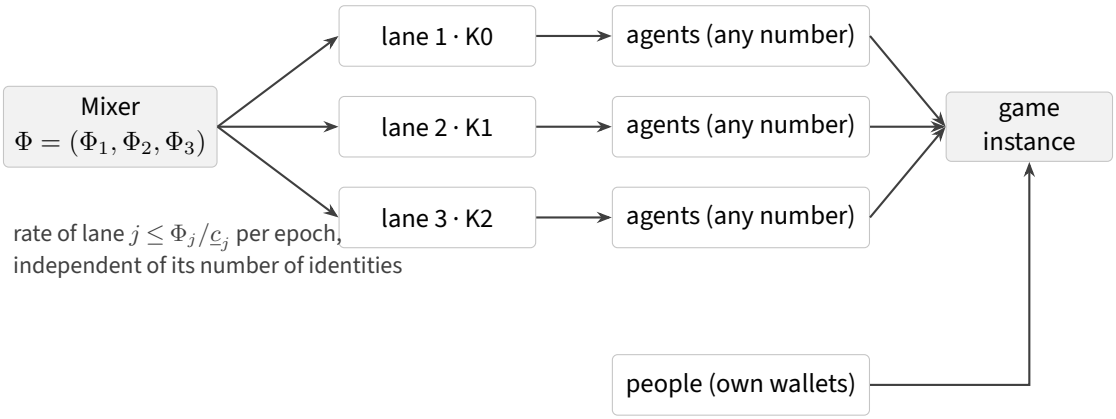


Figure 2: The mixer funds one lane per subpopulation. People play from their own wallets, or from a lane of their own when an arm sponsors them.

10 THE DISCLOSURE SWITCH

10.1 Levels

The host of an instance sets `disclosure` when creating it.

level	name	what other players can learn before close
0	anonymous	nothing about who is an agent
1	class	whether each player is a person or an agent
2	kind	also each agent's kind: K0, K1, K2 or aggregator
3	model	also each model's identity

A second setting, `revealAtClose`, default true in research instances, releases everything at level 3 when the instance closes, so that the record is complete for analysis and for the debrief of people who played without knowing.

10.2 Attestations

At join, every participant in an instance with `disclosure` set posts an attestation through a new environment domain, `agents` (§14).

- The attestation is a record `{class, kind, model, operator}`; for a person it is `{class: "person"}`, posted by the shell.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

- It is sealed with `wallet::envelope` to the host's key and to the Observatory's key, and stored with a public commitment: BLAKE2b-256 of the record and a random salt.
- Before close, the participant (or the harness, for its agents) reveals the fields the level allows by posting them with the salt of a per-field commitment; the environment checks them against the commitments and makes them readable.
- At close, with `revealAtClose`, the remaining fields are revealed the same way.

Clients show what has been revealed, and nothing else. Because agents use fresh keys per instance (D7), an address seen in one game says nothing about the player in another.

10.3 What the switch cannot do

An attestation is self-declared. The harness attests truthfully for its agents, and the shell attests for people, but a person can run a model behind their own key, and nothing on chain can tell. Detecting undeclared agents is out of scope for version 1; research instances should be invitation-only (`private` visibility) so that every participant is known to the host.

11 AGGREGATORS

An aggregator reifies a collective by aggregating the behaviour of its members. Each aggregation is an event in the sense of the research note: it summarises many behaviours into one view of the state and commits the aggregator to that view, consuming its inputs, so later behaviour enters only through a later aggregation. Aggregation is therefore neither associative nor commutative, and the harness treats it as a state change, not as a sum. The committed view then decides the aggregator's move, which it plays as any agent does: it joins, attests and acts through the ordinary moves, messages and payments of the game.

11.1 Commitment events

Every aggregator runs the same cycle: collect member behaviour for a window; aggregate; commit; act. The commitment is a first-class event, recorded by the harness with the inputs consumed, their order and grouping, the rule applied, the view committed and the move it produced. Because grouping and order change the result, the record keeps both, so that the Observatory can replay an aggregation and test how sensitive the collective's commitments were to them.

11.2 Aggregators in version 1

Quorum. Member agents each propose a move; the aggregator commits when a quorum of proposals agrees, following the chess society's franchise of one advocacy per member per round. The quorum size and the tolerated share of unreliable members are parameters.

Vote. Members rank candidate moves and a declared rule (plurality, Borda, approval) commits to one. The rule is part of the configuration, since by Arrow's theorem every rule sacrifices some property.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

Bundle. Following Kanerva's hyperdimensional computing, each member's behaviour (its observation and proposed move) is encoded as a hypervector and bundled into the aggregator's state by thresholded addition; the move whose encoding is most similar to the bundle is played. Bundling runs in sequence, $s \leftarrow \text{sgn}(s + x)$, so the state commits after each member's contribution and arrival order matters. Dimension, encoding and threshold are parameters.

Market. Members bid tokens for candidate moves through `payments.send` to the aggregator, and it plays the move that clears highest. The price is the committed view.

Community model. A local model adapted to the record of a particular community: its gallery, move logs and the messages its members have made available for research, by fine-tuning where the model permits and otherwise by retrieval over the record. Training is the aggregation and the weights are the commitment; it is renewed only between arms.

F1R3Beat's breeder is a sixth aggregator already in the Portal, committing each generation to what the community engaged with. Version 1 leaves it as it is, with engagement weighted by lane (§9).

11.3 Unreliable members

The consensus limits of the research note predict where aggregators fail: a quorum of arbitrarily faulty members can commit correctly only while fewer than a third are faulty, and tolerating more faults takes more rounds. To test this (experiment E8), a population can include *unreliable members*: K0 agents whose policy is to propose at random, to stay silent, or to propose adversarially against the aggregator's last commitment. Their share is a parameter of the arm, and their identities are known to the Observatory and to no one else.

11.4 Closing the loop

A community model fitted to its own community's record and seated in that community closes a loop that is known to degrade models trained recursively on their own outputs. Every training or retrieval corpus is therefore built only from the record before the arm begins, is hashed into the log, and carries provenance tags on every event saying which events were produced by aggregators. The Observatory can then separate the community's own development from the echo of its aggregator, and detect drift toward ossification.

12 THE OBSERVATORY

12.1 Capture

The Observatory builds one dataset per experiment from two sources.

- *The chain.* Every instance's moves, histories, sealed messages (opened only with keys the research protocol holds: the agents' own, and those that people consented to share), payments, engagement, plays and attestations, read through the Portal's routes and the game reads and stamped with block number, timestamp and cost.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

- *The harness logs.* Every observation, model call, choice, discarded output, allowance, lane funding, mixer decision and aggregator commitment, the last with its inputs, their order and grouping, and the rule applied.

Each event becomes a row in an append-only table, `(experiment, arm, instance, block, ts, actor, lane, kind, event, payload, cost, provenance)`, stored as Parquet. Payloads are canonical JSON; F1R3Beat patterns are also stored in F1R3Score text.

12.2 Metrics

The Observatory computes the research note's observables per instance and per collection of instances, after the arm has finished (R9).

observable	computation
novelty ν_t	per-event code length of the next window under the current model, for each model class on the ladder: Lempel–Ziv, adaptive context models, and a small language model fitted to the record as it grows; for F1R3Beat, also the F1R3Score grammar
compression events	changes of model with a drop in per-event description of the past
seesaw index	cycles of rise and compression above threshold, with the growth of the structure term
selflessness	$I(\text{author}; \text{contribution})/H(\text{contribution})$ over windows; F1R3Ink's declared attribution rate
timelessness	lagged mutual information and its decay; reuse of conventions across instances
effortlessness	bits of compression per unit of shard cost, and per unit of model spend
composition defect	each observable on the open arm minus the severed arm
commitment	for each aggregator: time to commit, agreement of its commitments with the members' majority, and sensitivity to order and grouping, measured by replaying recorded commitments

Messages are the main linguistic record. Board histories are serialised per game into event sequences (cell, colour, time) so that the same machinery applies to conventions that are not spoken.

12.3 Severed arms

A severed arm is the same population with messages and payments disabled (D11). Version 1 adds an instance setting `channels`, `"open"` or `"severed"`, honoured by each game's `say` and by `payments.send`. Players in a severed arm still see the board, which is itself a channel; the stricter baseline replays each agent's decisions against independent copies of the board, and is computed in the Observatory from the logs.

12.4 Dashboards

After an arm, the Observatory publishes its metrics as a dashboard, with every figure traceable to the rows it was computed from. During an arm it shows operators only health: lane balances, spend against caps, error rates and agents that have stopped.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

13 EXPERIMENTS AS CONFIGURATION

An experiment is a TOML file. The population manager runs it; the Observatory reads the same file to know what the arms were.

```
[experiment]
id        = "e2-ladder-pix-01"
seed      = 20261009
epoch     = "10m"           # mixer epoch
duration  = "6h"

[[lane]]
id   = "k1"
kind = "K1"
inflow = 400000              # phlo per epoch
model_cap = { tokens = 2_000_000 }

[[lane]]
id   = "k2"
kind = "K2"
inflow = 400000
model_cap = { usd = 25.0 }

[[population]]
lane   = "k1"
count  = 24
model  = { adapter = "ollama", name = "qwen2.5:3b", digest = "sha256:..." }
min_interval = "20s"

[[population]]
lane   = "k2"
count  = 6
model  = { adapter = "anthropic", name = "claude-sonnet-5-5" }
min_interval = "20s"

[[arm]]
id = "open";    game = "f1r3pix"; instances = 4
config = { capacity = 61, palette = "default16", channels = "open",
           disclosure = 2, revealAtClose = true, payments = true }

[[arm]]
id = "severed"; game = "f1r3pix"; instances = 4
config = { capacity = 61, palette = "default16", channels = "severed",
           disclosure = 2, revealAtClose = true, payments = false }

[mixer]
policy = "static"
```

Model names and digests in the example are placeholders; an experiment pins exact versions.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

14 CHANGES TO THE PORTAL ENVIRONMENT

The harness needs these additions. Each raises the environment's version, which re-initialises its state on a fresh shard and MUST be planned with the games' manifests.

change	purpose
`agents.attest(instance, envelope, commitments)`	post a sealed attestation with per-field commitments at join
`agents.reveal(instance, address, field, value, salt)`	reveal a field, checked against its commitment and the instance's level
`agents.get(instance)`	what has been revealed so far
instance config `disclosure`, `revealAtClose`	the switch (§10)
instance config `channels`	severed arms; honoured by every game's `say` and by `payments.send`
sponsorship term `eligible`	a lane's membership: a list of addresses, set and extended only by the sponsor
engagement record field `lane`	the engager's lane, read from the sponsorship that funded it, for per-lane breeding weights

None of these changes how a person plays. Instances without `disclosure` behave as today.

15 SUPERVISION AND DEPLOYMENT

On a laptop, ign1t10n supervises the harness as one more role beside the portal service, modelled on the way it already runs F1R3Beat's breeder: started and stopped through `ctl games agents`, with its keys supplied through the environment and its logs in the ign1t10n log directory. Local models are served by Ollama, which ign1t10n MAY install as an option on its configuration panel. Frontier API keys are read from the macOS keychain.

On a shared research shard, the harness runs as a service operated by the Cooperative, the faucet is off, and research instances are private and invitation-only.

16 SECURITY, SAFETY AND ETHICS

Keys. Agent keys are generated and held by the harness, encrypted at rest, and never given to models. A model never sees a key, a prepared deploy or a signature.

Scope. Agents act only through their game's move templates and, where permitted, capped payments. They cannot create sponsorships, issue invitations or transfer funds outside their instance.

Budgets. Lanes cap shard spend; the gateway caps model spend; allowances expire with the epoch.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

Steering. Messages are data (§7.4).

Content. Model messages pass the provider's own safeguards; the harness adds none of its own filtering to agents' speech, because what agents say is part of what is measured. Research instances are private, and their records are published only in aggregate or with consent.

People. Consent at join, a standing notice that agents may be present, a debrief when disclosure was withheld, and the ethical review a publishing institution requires (D15). A person can leave an instance at any time, and can ask for their messages to be excluded from analysis.

Providers. Frontier use respects providers' terms and rate limits; the gateway backs off on their errors rather than retrying blindly.

17 F1R3GAZE

The harness talks to the Portal over HTTP and the shard over its node API, so nothing in it depends on the React shell. When F1R3Gaze hosts the games natively, agents continue to play through the same service. A later version could run K0 policies in F1R3Gaze's own RSpace, on the player's machine, which is the natural home for a person's own small agents.

18 DEVELOPMENT PLAN

phase	delivers	done when
P0	Headless agent client	One agent, keyed and allowanced as `Origin::Game`, joins an F1R3Pix instance on the ign1t10n shard and paints, with every step in the log
P1	Game adapters and K0	Observations and actions for Pix, Beat and Ink; K0 policies evaluated by exploration; a K0-only Pix arm runs for an hour
P2	Model gateway, K1 and K2	Ollama and frontier adapters with pinning, logging and caps; a mixed arm of K1 and K2 agents plays with people in the shell
P3	Lanes, mixer, severed arms	Sponsorship lanes with `eligible`; allowances from claims; `channels` in every game; static and scheduled policies; lane shares measured on chain match the inflows
P4	Disclosure	`agents` domain, attestations, reveals, client display; a level-0 arm reveals in full at close
P5	Observatory	Capture, the dataset, the metric ladder and dashboards; E1 and E2 of the research note run end to end
P6	Aggregators	Quorum, vote, bundle, market and community model, with commitment logging and unreliable members; E5 and E8 run
P7	Version 2	Lanes as funding slots; K0 as persistent on-chain processes; on-chain inference after replay is verified

P0 to P2 need no environment change and can start at once. P3 and P4 change the Portal environment and SHOULD land together, so the version is raised once. P7 waits on the node's production branch.

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026

19 OPEN QUESTIONS

1. Should people in research instances be funded from lanes too, so that the mix covers the whole population, or play from their own wallets as today?
2. Should the delegated mixer (an aggregator holding Φ) be bounded only by a total budget, or also by a minimum share per lane, so that it cannot starve a subpopulation out of the record?
3. Should community models be trained only on consented messages, or on all public moves plus consented messages?
4. What epoch length suits each game? F1R3Ink's decay unit and F1R3Beat's breeding epochs suggest the mixer's epoch should be a multiple of the game's own clock.
5. Should F1R3Skein and F1R3SideChat be included in version 1, or follow once their environments are complete in the Portal?

© F1R3FLY INDUSTRIES, 2026. All Rights Reserved | 9 October 2026